

Type Theory — Homework Assignment

System F, System F_ω, and Monads

Due: Monday, April 6, in class or by email to the TA.

Notation. Throughout this assignment we work with the polymorphic λ -calculus (**System F**, also called $\lambda 2$). The symbol Π in a type expression denotes *universal quantification over types*, i.e. $\Pi \alpha : *. \sigma$ means $\forall \alpha : *. \sigma$. Both notations are used interchangeably. We write $*$ for the kind of all types and $\rightarrow$ for the function-type constructor.

Exercise 1 — Inhabitation in System F

Find System F terms that **inhabit** each of the following $\lambda 2$ (i.e. System F) types, in the given context Γ . In other words, for each item find a term M such that $\Gamma \vdash M : \sigma$ is derivable.

(a)

$$\sigma = \forall \alpha, \beta : *. (\mathbf{nat} \rightarrow \alpha) \rightarrow (\alpha \rightarrow \mathbf{nat} \rightarrow \beta) \rightarrow \mathbf{nat} \rightarrow \beta, \quad \Gamma \equiv \mathbf{nat} : *.$$

(b)

$$\sigma = \forall \delta : *. ((\alpha \rightarrow \gamma) \rightarrow \delta) \rightarrow (\alpha \rightarrow \beta) \rightarrow (\beta \rightarrow \gamma) \rightarrow \delta, \quad \Gamma \equiv \alpha : *, \beta : *, \gamma : *.$$

(c)

$$\sigma = \forall \alpha, \beta, \gamma : *. (\alpha \rightarrow (\beta \rightarrow \alpha) \rightarrow \gamma) \rightarrow \alpha \rightarrow \gamma, \quad \text{in the empty context } \Gamma = \emptyset.$$

Exercise 2 — The Bottom Type and a Typing Puzzle

Recall from the Curry–Howard perspective that, in System F, the *empty type* (logical falsehood) is encoded as

$$\perp \equiv \forall \alpha : *. \alpha.$$

A term of type $\perp$ would have to produce a value of *every* type, so $\perp$ is uninhabited in a consistent system. Nevertheless, *assuming* a variable $x : \perp$ we can build interesting terms.

Task. Let $\perp \equiv \forall \alpha : *. \alpha$ as above. Prove that the following term is **legal** (i.e. typable) in the empty context:

$$\lambda x : \perp. x (\perp \rightarrow \perp \rightarrow \perp) (x (\perp \rightarrow \perp) x) (x (\perp \rightarrow \perp \rightarrow \perp) x x).$$

Give a full derivation showing the type of each sub-expression and state the resulting type of the whole λ -abstraction.

Hint. Since $x : \forall \alpha. \alpha$, we may instantiate x at any type A to obtain $x A : A$. Work from the inside out: first type $x (\perp \rightarrow \perp)$, then $x (\perp \rightarrow \perp \rightarrow \perp)$, etc.

Exercise 3 — Kind Derivations in System F_ω

We now move to **System F_ω** ($\lambda\omega$), which extends System F with type-level operators (functions from types to types). Recall that $*$ is the kind of proper types and $\kappa_1 \rightarrow \kappa_2$ is the kind of type-level functions from kind κ_1 to kind κ_2 . The judgement form $\Gamma \vdash \sigma : \kappa$ means “in context Γ , the type expression σ has kind κ .”

Give **complete** $\lambda\omega$ -derivations **in tree (natural-deduction) format** — not abbreviated — of the following judgements. Use the kinding rules of System F_ω (axiom, variable, arrow-formation, abstraction, application).

(a) $\emptyset \vdash (* \rightarrow *) \rightarrow * : \Box$

Note: Here $\Box$ (“box”) is the sort that classifies kinds. The judgement asks you to show that $(* \rightarrow *) \rightarrow *$ is a well-formed kind.

(b) $\alpha : *, \beta : * \vdash (\alpha \rightarrow \beta) \rightarrow \alpha : *$

Exercise 4 — Logical Connectives via the Curry–Howard Isomorphism

Under the Curry–Howard correspondence, System F types are read as second-order propositional formulas:

$$A \wedge B \equiv \forall \gamma : *. (A \rightarrow B \rightarrow \gamma) \rightarrow \gamma, \quad A \vee B \equiv \forall \gamma : *. (A \rightarrow \gamma) \rightarrow (B \rightarrow \gamma) \rightarrow \gamma,$$

$$\perp \equiv \forall \alpha : *. \alpha, \quad \neg A \equiv A \rightarrow \perp.$$

Using the encodings above, find $\lambda 2$ -terms (System F terms) that represent the following logical operators. For each term, state its type and briefly justify its correctness.

(a) **Inclusive or** (disjunction introduction and elimination):

- $\text{inl} : A \rightarrow A \vee B$ (left injection),
- $\text{inr} : B \rightarrow A \vee B$ (right injection),
- $\text{case} : A \vee B \rightarrow (A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow C$ (case analysis).

(b) **Exclusive or** ($A \oplus B$).

Define

$$A \oplus B \equiv \forall \gamma : *. (A \rightarrow \neg B \rightarrow \gamma) \rightarrow (\neg A \rightarrow B \rightarrow \gamma) \rightarrow \gamma.$$

Provide at least the two introduction terms `xor-left` and `xor-right`.

(c) **Implication** ($A \Rightarrow B$).

Recall that in System F, implication is simply the arrow type $A \rightarrow B$. Write $\lambda 2$ -terms for:

- **Modus ponens:** a term of type $(A \rightarrow B) \rightarrow A \rightarrow B$.
 - The **S combinator:** a term of type $(A \rightarrow B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C$.
-

Exercise 5 — The Continuation Monad in System F

Motivation: from double negation to continuations

In Exercise 4 you encoded logical connectives as System F types. Recall that negation is $\neg A \equiv A \rightarrow \perp$. Now consider **double negation**: $\neg\neg A \equiv (A \rightarrow \perp) \rightarrow \perp$. A classical logician reads this as “if assuming A is false leads to a contradiction, then A must be true.” Constructively, however, $\neg\neg A$ is *weaker* than A . We cannot, in general, extract a witness of type A from a proof of $(A \rightarrow \perp) \rightarrow \perp$.

Something interesting happens if we replace the fixed $\perp$ by an *arbitrary* answer type R . The type $(A \rightarrow R) \rightarrow R$ no longer says “ A leads to a contradiction”; instead it says:

“Give me any *callback* $k : A \rightarrow R$ - a way to use a value of type A to produce an answer of type R
- and I will produce an R for you.”

This is exactly the pattern known as **continuation-passing style** (CPS): instead of computing a result of type A and returning it directly, a CPS computation takes an extra argument k (the **continuation**, i.e. “what to do next”) and passes its result to k .

CPS is ubiquitous in programming language theory, it is used in compilers as an intermediate representation, it explains control operators like exceptions, and more interestingly for us it gives a computational interpretation of classical logic (the **Curry–Howard correspondence for classical logic** due to Griffin, 1990).

The purpose of this exercise is to show that the type $(\alpha \rightarrow R) \rightarrow R$ forms a **monad**.

What is a monad? (recap)

A **monad** is a type constructor $M : * \rightarrow *$ together with two polymorphic operations:

$$\text{return} : \forall \alpha : *. \alpha \rightarrow M \alpha, \quad \text{bind} : \forall \alpha, \beta : *. M \alpha \rightarrow (\alpha \rightarrow M \beta) \rightarrow M \beta.$$

Think of $M \alpha$ as a “computation that when run produces a value of type α .” (but importantly we do not have to run the computation, we can talk about the computation without having to run it).

- **return** takes a pure value $a : \alpha$ and wraps it as a trivial computation that just produces a .
- **bind** takes a computation $m : M \alpha$ and a function $f : \alpha \rightarrow M \beta$ (which says “given the computation described by m , here is how to continue”), and chains them into a single computation of type $M \beta$.

These usually are expected to satisfy three **monad laws** (see part (d) below), which ensure that **return** is a neutral element for sequencing and that sequencing is associative, much like how function composition is associative.

The continuation monad

Fix a type $R : *$ once and for all (the “answer type”, the type of the final result of the whole program). Define the type operator

$$\text{Cont}_R \alpha \equiv (\alpha \rightarrow R) \rightarrow R.$$

A term $m : \text{Cont}_R \alpha$ is a CPS computation. The α is never explicitly available (as a concrete value) throughout the computation; it only exists as an argument that m pipes through the callback.

Example (informal). Suppose $R = \text{nat}$ and $\alpha = \text{nat}$. The term $\lambda k : \text{nat} \rightarrow \text{nat}. k \ 5$ has type $(\text{nat} \rightarrow \text{nat}) \rightarrow \text{nat}$. When you run this computation by supplying a callback k , the value 5 is fed into k . The computation does not return 5; it returns $k \ 5$. Given the callback $k = \lambda n. n + 1$, it evaluates to $k \ 5 = 6$.

Running a continuation. To extract a final answer from a CPS computation $m : \text{Cont}_R \alpha = (\alpha \rightarrow R) \rightarrow R$, you apply it to a callback. We give this a name:

$$\text{runCont} : \forall \alpha : *. ((\alpha \rightarrow R) \rightarrow R) \rightarrow (\alpha \rightarrow R) \rightarrow R \quad \text{with} \quad \text{runCont } m \ k \equiv m \ k.$$

This is just function application. But naming it makes explicit that every CPS computation remains inert until you supply a callback. In the special case where $\alpha = R$, you can use the identity $\lambda a : R. a$ as the callback: $\text{runCont } m \ (\lambda a. a)$ simply extracts the final answer.

All the examples below use this implicitly: when we say “run with callback k ” we mean $\text{runCont } m \ k$, i.e. $m \ k$.

Syntactic sugar: $>>=$ and **do**-notation

Writing $\text{bind } \alpha \beta m f$ everywhere is heavy. Two shorthands are standard:

1. **Infix bind** ($>>=$). We write $m >>= f$ for $\text{bind } m f$ (suppressing the type arguments α, β , which can be inferred).
2. **do-notation**. A block of the form

$$\mathbf{do} \{ x \leftarrow m; e \}$$

is syntactic sugar for $m >>= (\lambda x. e)$, i.e. “run m , name its result x , then continue with e .” Multiple lines are nested:

$$\mathbf{do} \{ x \leftarrow m_1; y \leftarrow m_2; \mathbf{return} (x + y) \} \equiv m_1 >>= \lambda x. m_2 >>= \lambda y. \mathbf{return} (x + y).$$

An expression without $\leftarrow$ is just sequenced: $\mathbf{do} \{ m; e \} \equiv m >>= \lambda_. e$.

These are purely notational, no new typing rules are needed. The exercises ask for the raw System F terms, but the sugar makes the *intent* of the examples below clearer.

Worked examples

The following examples all use $R = \mathbf{nat}$.

Example 1: **return** and $>>=$.

$$\mathbf{return} 5 >>= \lambda n. \mathbf{return} (n + 10)$$

Read: “take the value 5, then add 10.” The result is a CPS computation that, when run with a callback k , feeds 15 into k . For instance, with $k = \lambda n. n \times 2$, we get 30.

Example 2: a multi-step pipeline in **do**-notation.

$$\mathbf{do} \{ x \leftarrow \mathbf{return} 3; y \leftarrow \mathbf{return} (x + 10); \mathbf{return} (y \times 2) \}$$

Read: “start with 3, add 10 to get 13, double to get 26.” This desugars to:

$$\mathbf{return} 3 >>= \lambda x. \mathbf{return} (x + 10) >>= \lambda y. \mathbf{return} (y \times 2)$$

Each $>>=$ feeds the value held by the left-hand computation into the function on the right. When run, 26 is fed into the callback.

Example 3: **early exit** with **calcc**.

The operation **calcc** (defined formally in part (c) below) takes a function as its argument. The first parameter of that function, let us call it **abort**, is itself a function that serves as an escape mechanism. When applied to a value $a : \alpha$, **abort** a yields a term of type

$$\mathbf{abort} a : \forall \beta : *. (\beta \rightarrow R) \rightarrow R.$$

Why $\forall \beta$? Because after escaping, the surrounding code never runs, so the type system must allow **abort** a to stand in for a computation of *any* type β .

A note about $>>=$. In **do**-notation we write **abort** 99 as though it has a single concrete monadic type, but $>>=$ needs its first argument at a *specific* type $(\beta \rightarrow R) \rightarrow R$, not a polymorphic one. In raw System F this is resolved by explicit type application: you write **abort** 99 β to instantiate $\forall \beta$ at the type required by that particular $>>=$. The **do**-notation does not show this step because the compiler performs type inference and

inserts the correct type application automatically. But in System F there is no inference, every $\forall$ must be explicitly consumed by a type application written by hand.

The result of `callcc` ($\lambda \text{abort}. \dots$) is still a value in the continuation monad, that is, it has type $(\alpha \rightarrow R) \rightarrow R$. Like any CPS computation, you extract a final answer by applying `runCont` with a callback.

`callcc` ($\lambda \text{abort}. \text{do } \{ x \leftarrow \text{return } 42; \text{ abort } 99; \text{return } (x \times 2) \}$)

Read this as: `callcc` receives a function whose parameter is `abort` (the escape function). Inside the body:

- `x ← return 42`: bind $x = 42$.
- `abort 99`: call the escape function with value 99. (In raw System F: `abort 99 nat`, instantiating $\forall\beta$ at `nat` to match the surrounding context.) This **jumps straight to the outer callback**, skipping everything after it.
- `return (x × 2)`: **never reached**.

If the outer callback is $\lambda n. n + 1$, the final answer is $99 + 1 = 100$, not $42 \times 2 + 1 = 85$.

Now compare with a version that does *not* abort:

`callcc` ($\lambda \text{abort}. \text{do } \{ x \leftarrow \text{return } 42; \text{return } (x \times 2) \}$)

Here `abort` is available but never called, so execution proceeds normally. When run with callback $\lambda n. n + 1$, the value 84 is fed into it, giving 85.

The point: `callcc` gives you a “goto” that jumps to a fixed point in the computation, discarding everything in between. The body of the computation must be written *inside* the function passed to `callcc`, that is where `abort` is in scope.

Tasks

Throughout this exercise, $R : *$ is a fixed type variable assumed to be in the context: $\Gamma \equiv R : *$.

(a) Write a System F term for `return`.

Its type should be

$$\text{return} : \forall \alpha : *. \alpha \rightarrow (\alpha \rightarrow R) \rightarrow R.$$

The idea: given a value $a : \alpha$ that you already have, and a continuation $k : \alpha \rightarrow R$ that is waiting for an α , the simplest thing to do is to hand a to k .

Hint. The term has the shape $\Lambda \alpha : *. \lambda a : \alpha. \lambda k : \alpha \rightarrow R. \dots$

(b) Write a System F term for `bind`.

Its type should be

$$\text{bind} : \forall \alpha, \beta : *. \underbrace{((\alpha \rightarrow R) \rightarrow R)}_m \rightarrow \underbrace{(\alpha \rightarrow (\beta \rightarrow R) \rightarrow R)}_f \rightarrow \underbrace{(\beta \rightarrow R)}_k \rightarrow R.$$

Read the three arguments as a pipeline:

- $m : (\alpha \rightarrow R) \rightarrow R$ is a CPS computation. Its output type is R , not α ! The α never “comes out” of m . When you run m by feeding it a callback, m pipes a value of type α through that callback. That is the essence of CPS: m “has an α ” only in the sense that it will feed one into whatever callback it receives.
- $f : \alpha \rightarrow (\beta \rightarrow R) \rightarrow R$ is the “next step.” It expects to receive an α (the one that m will pass to its callback), and returns a new CPS computation, one that will in turn pass a β to *its* continuation.

- $k : \beta \rightarrow R$ is the *outermost* callback. It is not itself a CPS computation — it is what you feed *to* the combined computation $\text{bind } m \ f : (\beta \rightarrow R) \rightarrow R$ in order to extract the final answer R . Think of k as the very last step: once a β is available, k turns it directly into the answer.

Your job is to wire these together.

Hint. The body of the term is $m (\lambda a : \alpha. f \ a \ k)$

(c) Write a System F term for `callcc` (call-with-current-continuation).

`callcc` is the key operation that makes CPS interesting from a control-flow perspective. It gives the programmer access to the “current continuation”, i.e. a snapshot of “what the rest of the program is about to do”, and the ability to *abort* the normal flow by jumping directly to that continuation.

Its type should be

$$\text{callcc} : \forall \alpha : *. \underbrace{\left((\alpha \rightarrow \forall \beta : *. (\beta \rightarrow R) \rightarrow R) \right)}_{\text{escape function}} \rightarrow \underbrace{(\alpha \rightarrow R \rightarrow R)}_{\text{body computation}} \rightarrow (\alpha \rightarrow R) \rightarrow R.$$

Read this as follows. `callcc` takes two arguments:

1. A function f , whose first parameter has type $\alpha \rightarrow \forall \beta : *. (\beta \rightarrow R) \rightarrow R$. The label “escape function” in the type is just a description of the *role* this parameter plays — it is not something you define separately. Your job is to figure out what concrete argument to pass to f in this position. The intended behaviour: if f ever applies its first parameter to some $a : \alpha$, the whole computation should short-circuit and deliver a to the outer callback, ignoring everything else. The $\forall \beta$ quantification allows this escape to stand in for a computation of any type β (since it never actually needs to produce a β — it escapes before that). The second parameter of f is the current callback $k : \alpha \rightarrow R$.
2. A callback $k : \alpha \rightarrow R$.

f must return a computation $(\alpha \rightarrow R) \rightarrow R$, which `callcc` then runs with k .

Hint. Your term will have the shape $\Lambda \alpha : *. \lambda f : \dots \lambda k : \alpha \rightarrow R. f \ (\dots) \ k$. The challenge is to construct the first argument of f , a term of type $\alpha \rightarrow \forall \beta. (\beta \rightarrow R) \rightarrow R$, using only k . Think about what this term should do: given an $a : \alpha$ and any local callback of type $\beta \rightarrow R$, it should ignore that local callback and instead apply k to a .

(d) (*Bonus.*) Verify on paper that your `return` and `bind` satisfy the three monad laws by carrying out the β -reductions explicitly:

$$\begin{aligned} \text{bind } (\text{return } a) \ f &=_{\beta} f \ a && \text{(left identity),} \\ \text{bind } m \ \text{return} &=_{\beta} m && \text{(right identity),} \\ \text{bind } (\text{bind } m \ f) \ g &=_{\beta} \text{bind } m \ (\lambda a. \text{bind } (f \ a) \ g) && \text{(associativity).} \end{aligned}$$

Hint for right identity. After substitution you should obtain $\lambda k. m \ (\lambda a. k \ a)$. Argue that $\lambda a. k \ a =_{\eta} k$, so this equals $\lambda k. m \ k =_{\eta} m$. Here $=_{\eta}$ means that we identify two functions f and g when $fx = gx$ is true for all x .

End of assignment.